

NOM Prénom :
Nunéro Étudiant :

Université Paris Cité

Année 2022–2023

Programmation Réseaux - Partiel

Partie sur les threads

Aucun document n'est autorisé

Lorsque vous aurez à modifier du code, vous pourrez juste écrire les lignes à modifier en précisant leur numéro ou les lignes à ajouter en précisant entre quelles lignes elles s'insèrent.

1. Soit le code suivant tel que `serve` soit une fonction de prototype `void *serve(void *)` envoyant un message sur la socket passée en argument :

```
1 int tour = 0;  
2  
3 while(tour < 10){  
4     int sock2 = accept(sock , NULL, NULL);  
5     pthread_t thread;  
6  
7     if (pthread_create(&thread , NULL, serve , &sock2))  
8         continue;  
9     tour++;  
10 }
```

(a) Quels problèmes peut-on rencontrer lors de l'exécution de ce code ?

(b) Et comment corriger cela ?

2. Soit le code suivant :

```
1 void *serve(void *arg) {
2     int *var = (int *) arg;
3     *var *= 2;
4     int entier = *var;
5     entier++;
6     printf("valeur : %d\n", entier);
7     entier--;
8     int f = open("fic.txt", O_WRONLY | O_APPEND | O_CREAT, 0666);
9     if(f < 0) exit(1);
10    write(f, &entier, sizeof(entier));
11    close(f);
12    return NULL;
13 }
14
15 int main(int argc, char *argv[]) {
16     int tour = 0;
17     int *a = malloc(sizeof(int));
18     *a = 1;
19
20     while(tour < 10){
21         pthread_t thread;
22
23         if (pthread_create(&thread, NULL, serve, a))
24             continue;
25         tour++;
26     }
27     sleep(10);
28
29     return 0;
30 }
```

- (a) Lors de l'exécution de ce code, que se passe-t-il si l'appel système à `open` en ligne 8 échoue ?
- (b) Pourquoi a-t-on mis l'instruction `sleep(10)` en ligne 27 ?
- (c) Modifier le programme afin qu'il permette d'obtenir un résultat équivalent sans faire appel à l'instruction `sleep(10)`.
- (d) Peut-on connaître le contenu du fichier `fic.txt` après exécution du code ? Expliquez
- (e) Déterminez les sections critiques de ce code.
- (f) Corrigez le programme, afin qu'après exécution, le fichier `fic.txt` contienne toutes les valeurs prises par la variable `a`, à l'exception de la valeur 1.
- (g) Donnez alors toutes les valeurs prises par la variable `a`.

- (h) Modifiez à nouveau le programme afin qu'il prenne comme unique argument de la ligne de commande le nom du fichier et qu'il passe ensuite la variable `a` et ce nom en paramètre de la fonction `serve` afin qu'elle l'utilise dans l'appel à `open`.

Prototypes de fonctions pouvant être utiles

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
int close(int fd);
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
pid_t fork(void);
void freeaddrinfo(struct addrinfo *res);
int listen(int sockfd, int backlog);
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine) (void *), void *arg);
int pthread_join(pthread_t thread, void **retval);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
const char *inet_ntop(int af, const void *src, char *dst, socklen_t size);
int inet_pton(int af, const char *src, void *dst);
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
int socket(int domain, int type, int protocol);
pid_t waitpid(pid_t pid, int *wstatus, int options);
```